color_dct_engine.v, explained part by part

JPEG-style colour image compression engine in Verilog, DE2-115 FPGA project. Prepared by Dev Vijay for the project review.

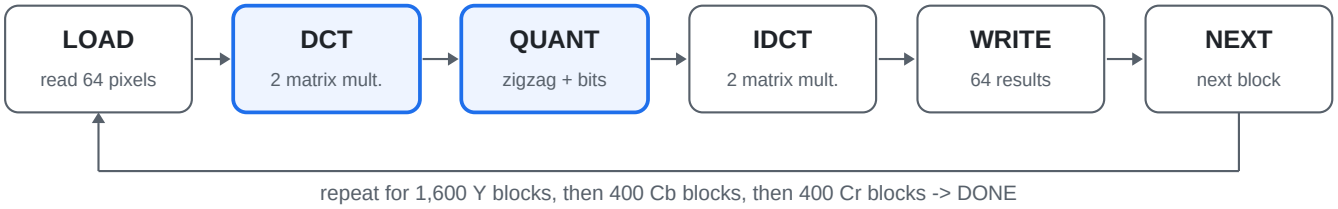
1. What the module does, in one paragraph

color_dct_engine takes the 320 x 320 colour picture stored in the original-image memory, compresses it the way JPEG does, and writes a decompressed copy into three result memories so the VGA can show it. While compressing, it also counts how many bits the compressed picture would need (**total_bits**), which the display module turns into the *% memory saved*. One full picture takes about **5.5 million clock cycles = 0.22 s at 25 MHz**.

The four ideas behind it

- **Split brightness from colour.** Each RGB pixel becomes Y (brightness), Cb (blue colour) and Cr (red colour). Cb and Cr are kept at half width and half height (4:2:0 subsampling), because the eye notices colour detail far less.
- **Work on 8 x 8 blocks.** Y is 40 x 40 = 1,600 blocks; Cb and Cr are 20 x 20 = 400 blocks each.
- **DCT.** Each block of 64 pixel values is converted into 64 frequency values. The first (DC) is the average brightness; the rest describe finer and finer detail.
- **Quantization.** Every frequency value is divided by a number from the JPEG table and rounded. Most detail values become 0. This is where data is thrown away, and why the result looks slightly different from the original.

Work done on every 8 x 8 block (highlighted = where compression happens)



Inputs and outputs at a glance

Port	Direction	Meaning
clk, reset	in	25 MHz clock; reset (KEY0 or a start pulse) restarts the whole picture
quality[1:0]	in	SW1 SW0: quantization table x1, x2, x4 or x8
rd_addr[16:0]	out	address of the pixel the engine wants to read from orig_mem
rd_rgb[15:0]	in	that pixel, RGB565, arriving one clock later
wr_addr, wr_data, we	out	write one decompressed value into a result memory
wr_chan[1:0]	out	which result memory: 0 = Y, 1 = Cb, 2 = Cr
done	out	1 when all three channels are finished
total_bits[23:0]	out	compressed size of Y + Cb + Cr in bits

Time budget (why it takes 0.22 s)

State	Cycles per Y block	Cycles per Cb/Cr block	Why
LOAD	65	257	64 pixels (Y) or 256 pixels averaged to 64 (Cb/Cr), +1 for memory delay
MUL (DCT)	1,024	1,024	2 matrix multiplies x 64 outputs x 8 multiply-adds
QUANT	64	64	one coefficient per clock
MUL (IDCT)	1,024	1,024	2 more matrix multiplies
WRITE + NEXT	65	65	64 writes + 1 clock to pick the next block
Total	**2,242**	**2,434**	1,600 x 2,242 + 800 x 2,434 = 5,534,400 cycles

2. The code, part by part

Each part below shows the real code from the file (line numbers on the left of each heading), then what it does and why it is written that way.

Part A - Module header and ports (lines 28-40)

```
module color_dct_engine (
    input  wire        clk,
    input  wire        reset,
    input  wire [1:0]  quality,     // 0 = light ... 3 = heavy compression
    output reg  [16:0] rd_addr,     // pixel address in the original image
    input  wire [15:0] rd_rgb,      // RGB565 pixel (arrives 1 clock later)
    output reg  [16:0] wr_addr,     // address in the result memory
    output reg  [7:0]  wr_data,     // value to write
    output reg         we,          // write enable
    output reg  [1:0]  wr_chan,     // which result memory: 0 = Y, 1 = Cb, 2 = Cr
    output reg         done,        // 1 when all 3 channels are finished
    output reg  [23:0] total_bits   // compressed size of Y + Cb + Cr in bits
);
```

This is the engine's connection to the outside. It never touches the VGA or the camera directly: it only reads pixels by address (**rd_addr / rd_rgb**) and writes results by address (**wr_addr / wr_data / we / wr_chan**). That is why the same file works unchanged in both the colour project (stored image) and the camera project.

- **rd_rgb arrives one clock after rd_addr**, because FPGA block RAM is synchronous. Part F handles this delay.
- **wr_chan** tells the top module which of the three result memories to write.
- **total_bits** is 24 bits wide: the worst case is under 2.5 million, which fits in 2^{24} = 16.7 million.

Part B - States and working registers (lines 42-64)

```
    // ---------------- states ----------------
    localparam S_START = 3'd0,
               S_LOAD  = 3'd1,
               S_MUL   = 3'd2,
               S_QUANT = 3'd3,
               S_WRITE = 3'd4,
               S_NEXT  = 3'd5,
               S_DONE  = 3'd6;

    reg [2:0] state;
    reg [1:0] chan;          // 0 = Y, 1 = Cb, 2 = Cr
    reg [1:0] mode;          // which matrix multiply (0..3)
    reg [8:0] cnt;           // step counter inside a state
    reg [5:0] bx, by;        // block column / row
    reg [1:0] shift;         // quality latched at start
    reg signed [31:0] acc;   // multiply-accumulate register
    reg signed [12:0] prev_dc;   // DC value of the previous block
    reg [5:0]         run;       // zeros seen since the last non-zero

    reg signed [15:0] bufA [0:63];   // 8x8 working buffer A
    reg signed [15:0] bufB [0:63];   // 8x8 working buffer B

    wire is_y = (chan == 2'd0);
```

Register	Width	Purpose
state	3 bits	which step the state machine is in (7 states)
chan	2 bits	which channel is being compressed: 0 = Y, 1 = Cb, 2 = Cr
mode	2 bits	which of the 4 matrix multiplies MUL is doing
cnt	9 bits	step counter inside a state; up to 511 for the multiplies
bx, by	6 bits	block column and row (0..39 for Y, 0..19 for Cb/Cr)
shift	2 bits	quality switches copied at the start, so moving SW1 SW0 mid-picture changes nothing
acc	32 bits signed	multiply-accumulate sum for one DCT/IDCT output
prev_dc	13 bits signed	DC value of the previous block (JPEG stores the DC as a difference)
run	6 bits	how many zero coefficients came in a row (for run-length counting)
bufA, bufB	64 x 16 bits	two 8 x 8 working buffers; data moves A -> B -> A during the transforms
is_y	wire	shortcut: 1 when compressing Y, 0 for Cb/Cr

bufA and bufB are register arrays, not block RAM, because the multiply step reads them at a different position every clock without waiting.

Part C - Constant tables (lines 66-192)

The tables are written as Verilog **functions** containing a **case** statement. Quartus turns each one into a small ROM made of logic. The index {u, i} means row u, column i of an 8 x 8 table, packed into 6 bits.

Excerpt, lines 68-88 (shortened)

```
    function signed [13:0] coef;          // DCT matrix C[u][i] x 4096
        input [5:0] n;
        case (n)
            6'd0:  coef =  14'sd1448; ...      // row u = 0: all 1448
            6'd8:  coef =  14'sd2009; 6'd9:  coef =  14'sd1703; 6'd10: coef =  14'sd1138; ...
            ...
        endcase
    endfunction
```

Function	What it holds	Where the numbers come from
coef(n)	8 x 8 DCT matrix, scaled by 4096	C[u][i] = c(u) cos((2i+1) u pi / 16), with c(0) = sqrt(1/8), c(u) = sqrt(2/8). Example: C[0][i] = 0.35355 x 4096 = 1448
qtab(n)	JPEG luminance quantization table (for Y)	Standard table from the JPEG standard (16, 11, 10, 16, ...)
rtab(n)	65536 / qtab(n)	Reciprocal, so 'divide by Q' becomes 'multiply then shift right 16'. 65536/16 = 4096
qtab_c, rtab_c	JPEG chrominance table and its reciprocals (for Cb, Cr)	Small values top-left, 99 everywhere else
zigzag(p)	position 0..63 in zigzag order -> index in the block	Standard JPEG zigzag: 0, 1, 8, 16, 9, 2, ...
nbits(v)	how many bits are needed to write v	Loop finds the highest 1 bit: 5 (101) -> 3 bits

Why 4096? FPGAs work best with whole numbers. Multiplying the cosines by 4096 (= 2^{12}) keeps about 3-4 correct decimal digits, and dividing back by 4096 is just a shift of 12 bits.

nbits, lines 159-168

```
    // how many bits are needed to write the number v (0 -> 0, 1 -> 1, 2..3 -> 2, 4..7 -> 3, ...)
    function [3:0] nbits;
        input [13:0] v;
        integer t;
        begin
            nbits = 4'd0;
            for (t = 0; t < 14; t = t + 1)
                if (v[t]) nbits = t + 1;
        end
    endfunction
```

Part D - RGB565 to Y, Cb, Cr (lines 194-212)

```
    // ---------------- RGB565 -> Y / Cb / Cr ----------------
    wire [7:0] r8 = {rd_rgb[15:11], rd_rgb[15:13]};   // 5/6/5 bits -> 8 bits
    wire [7:0] g8 = {rd_rgb[10:5],  rd_rgb[10:9]};
    wire [7:0] b8 = {rd_rgb[4:0],   rd_rgb[4:2]};
    wire signed [17:0] rs = {10'd0, r8};
    wire signed [17:0] gs = {10'd0, g8};
    wire signed [17:0] bs = {10'd0, b8};

    // x256 integer versions of the JPEG colour formulas
    wire signed [17:0] y_t  =  18'sd77  * rs + 18'sd150 * gs + 18'sd29  * bs;
    wire signed [17:0] cb_t = -18'sd43  * rs - 18'sd85  * gs + 18'sd128 * bs;
    wire signed [17:0] cr_t =  18'sd128 * rs - 18'sd107 * gs - 18'sd21  * bs;

    wire signed [17:0] y_r  = (y_t  + 18'sd128) >>> 8;      // 0..255
    wire signed [17:0] cb_r = (cb_t + 18'sd128) >>> 8;      // about -128..127 (= Cb - 128)
    wire signed [17:0] cr_r = (cr_t + 18'sd128) >>> 8;      // about -128..127 (= Cr - 128)

    wire signed [15:0] y_val = $signed(y_r[15:0]) - 16'sd128;              // level shift
    wire signed [15:0] c_val = (chan == 2'd1) ? $signed(cb_r[15:0]) : $signed(cr_r[15:0]);
```

- **Lines 195-197** expand 5/6/5-bit colour back to 8 bits by repeating the top bits (e.g. red 11111 -> 11111111 = 255, so full red stays full red).
- **Lines 203-205** are the JPEG colour formulas multiplied by 256 so they use whole numbers: Y = 0.299R + 0.587G + 0.114B becomes (77R + 150G + 29B) / 256.
- **Lines 207-209**: adding 128 then shifting right by 8 divides by 256 with rounding. **>>>** is an arithmetic shift that keeps the sign, needed because Cb and Cr can be negative.
- **Line 211 (level shift)**: Y is 0..255; subtracting 128 makes it -128..127, centred on zero. The DCT then gives a small DC value for mid-gray, as in real JPEG. Cb and Cr already come out centred on zero.
- **Line 212** picks Cb or Cr depending on which pass is running.

Part E - Address generation (lines 214-230)

```
    // ---------------- addresses ----------------
    // Y pass   : 8x8 pixels per block,   pixel (i,j) = cnt[5:3], cnt[2:0]
    // Cb/Cr pass: 16x16 pixels per block, pixel (i,j) = cnt[7:4], cnt[3:0]
    wire [8:0] y_row = {by, 3'b000}       + {6'd0, cnt[5:3]};
    wire [8:0] y_col = {bx, 3'b000}       + {6'd0, cnt[2:0]};
    wire [8:0] p_row = {by[4:0], 4'b0000} + {5'd0, cnt[7:4]};
    wire [8:0] p_col = {bx[4:0], 4'b0000} + {5'd0, cnt[3:0]};
    wire [8:0] rrow  = is_y ? y_row : p_row;
    wire [8:0] rcol  = is_y ? y_col : p_col;

    always @(*) rd_addr = ({8'd0, rrow} << 8) + ({8'd0, rrow} << 6) + {8'd0, rcol};   // row*320+col

    // write addresses: Y image is 320 wide, Cb/Cr images are 160 wide
    wire [16:0] y_wr_addr = ({8'd0, y_row} << 8) + ({8'd0, y_row} << 6) + {8'd0, y_col};
    wire [7:0]  c_row     = {by[4:0], 3'b000} + {5'd0, cnt[5:3]};       // 0..159
    wire [7:0]  c_col     = {bx[4:0], 3'b000} + {5'd0, cnt[2:0]};
    wire [16:0] c_wr_addr = ({9'd0, c_row} << 7) + ({9'd0, c_row} << 5) + {9'd0, c_col};  // row*160+col
```

- **Y pass:** 8 x 8 pixels per block. cnt[5:3] is the row inside the block and cnt[2:0] the column, so counting cnt from 0 to 63 walks the block row by row.
- **Cb/Cr pass:** one 8 x 8 colour block covers **16 x 16 pixels** of the picture (half size), so cnt[7:4] and cnt[3:0] walk 256 pixels.

- **{by, 3'b000}** means by x 8: adding zeros on the right multiplies by a power of two for free.
- **row x 320** is done as (row << 8) + (row << 6) = 256 row + 64 row. No multiplier needed.
- Result images: Y is 320 wide (x 320); Cb/Cr are 160 wide, so **row x 160 = (row << 7) + (row << 5)**.
- **rd_addr** is combinational (always @(*)), so the memory sees the new address in the same clock that cnt changes.

Part F - LOAD helpers: the 2 x 2 colour average (lines 232-237)

```
// ---------------- LOAD helpers ----------------
wire [5:0] cnt_m1 = cnt[5:0] - 6'd1;              // Y: data arriving now is pixel cnt-1
wire [7:0] p_m1   = cnt[7:0] - 8'd1;              // Cb/Cr: pixel cnt-1 of the 16x16 area
wire [5:0] c_idx  = {p_m1[7:5], p_m1[3:1]};       // colour sample = (row/2, col/2)
wire signed [15:0] c_acc = bufA[c_idx] + c_val;   // running sum of the 2x2 group
wire signed [15:0] c_avg = (c_acc + 16'sd2) >>> 2; // average of 4
```

- **cnt_m1 = cnt - 1**: because the memory answers one clock late, the pixel arriving now belongs to the previous count. LOAD therefore runs cnt = 0..64 and stores pixel cnt-1.
- **c_idx** maps a pixel (row r, column c inside the 16 x 16 area) to its colour sample (r/2, c/2): it takes bits [7:5] and [3:1], i.e. drops the lowest bit of row and column.
- **c_acc** adds the new pixel to the running sum; **c_avg** = (sum + 2) / 4 with rounding, using a shift.

Part G - The matrix-multiply datapath, the heart of the DCT (lines 239-262)

```
// ---------------- matrix multiply datapath ----------------
//   mode 0 : bufB = C  * bufA       (DCT, step 1)
//   mode 1 : bufA = bufB * C'       (DCT, step 2)
//   mode 2 : bufB = C' * bufA       (IDCT, step 1)
//   mode 3 : bufA = bufB * C        (IDCT, step 2)
wire [2:0] a = cnt[8:6];
wire [2:0] b = cnt[5:3];
wire [2:0] k = cnt[2:0];

reg signed [13:0] c_op;
reg signed [15:0] d_op;
always @(*) begin
    case (mode)
        2'd0:    begin c_op = coef({a, k}); d_op = bufA[{k, b}]; end
        2'd1:    begin c_op = coef({b, k}); d_op = bufB[{a, k}]; end
        2'd2:    begin c_op = coef({k, a}); d_op = bufA[{k, b}]; end
        default: begin c_op = coef({k, b}); d_op = bufB[{a, k}]; end
    endcase
end

wire signed [29:0] prod     = d_op * c_op;
wire signed [31:0] acc_next = (k == 3'd0) ? prod : acc + prod;
wire signed [31:0] acc_rnd  = acc_next + 32'sd2048;
wire signed [15:0] mul_out  = acc_rnd[27:12];              // divide by 4096
```

The 2-D DCT of a block X is **F = C x X x C^T**, and the inverse is **X = C^T x F x C**. Each is two ordinary 8 x 8 matrix multiplies, so the engine has **one multiply-accumulate unit** reused in four modes:

mode	Computes	Reads	Writes	Step
0	T = C x X	bufA (pixels)	bufB	DCT, rows
1	F = T x C^T	bufB	bufA (frequencies)	DCT, columns
2	S = C^T x F	bufA (dequantized)	bufB	IDCT, rows
3	X' = S x C	bufB	bufA (pixels again)	IDCT, columns

- **cnt splits into {a, b, k}**: a = output row, b = output column, k = term in the sum. One output = sum over k = 0..7 of 8 products, so 64 outputs x 8 = 512 clocks per mode.
- **prod** = one 16-bit value x one 14-bit constant = 30-bit product, done by the FPGA's built-in multiplier.
- **acc_next** restarts the sum when k = 0, otherwise adds to it.
- **acc_rnd** adds 2048 (half of 4096) and **mul_out = bits [27:12]** divides by 4096 with rounding, removing the x 4096 scale of the constants.

- Ping-pong between bufA and bufB means a mode never overwrites the data it is still reading.

Why two 1-D passes instead of a direct 2-D formula? The direct way needs 64 multiplies per output (4,096 per block); the separable way needs 16 per output (1,024 per block), 4 times less work.

Part H - Quantize and dequantize (lines 264-275)

```
    // ---------------- quantize + dequantize (zigzag order) ----------------
    wire [5:0]         zz_n   = zigzag(cnt[5:0]);
    wire [6:0]         q_tab  = is_y ? qtab(zz_n) : qtab_c(zz_n);
    wire [12:0]        r_tab  = is_y ? rtab(zz_n) : rtab_c(zz_n);
    wire signed [15:0] q_in   = bufA[zz_n];
    wire               q_neg  = q_in[15];
    wire [15:0]        q_mag  = q_neg ? -q_in : q_in;
    wire [31:0]        q_num  = q_mag * r_tab + (32'd32768 << shift);
    wire [31:0]        q_val  = q_num >> (5'd16 + shift);          // round(|F| / Q)
    wire [15:0]        q_step = {9'd0, q_tab} << shift;
    wire [31:0]        dq     = q_val * q_step;
    wire signed [15:0] dq_out = q_neg ? -dq[15:0] : dq[15:0];
```

- **zz_n**: coefficients are visited in zigzag order (low frequencies first), so the zeros bunch at the end.
- **q_tab / r_tab** choose the luminance table for Y and the chrominance table for Cb/Cr.
- **q_mag**: work on the magnitude (always positive), and remember the sign in q_neg.
- **q_num / q_val**: |F| x (65536/Q) >> 16 = |F| / Q. Adding 32768 (half of 65536) first makes it round to nearest. Both shifts grow by **shift**, so SW = 01, 10, 11 divide by 2Q, 4Q, 8Q with no extra tables.
- **dq** multiplies back by the step: this is the value a JPEG decoder would rebuild. The detail lost in rounding is gone forever, which is the 'lossy' part.
- **dq_out** puts the sign back.

Why multiply by a reciprocal? A hardware divider is large and slow; a multiplier is already built into the FPGA and finishes in one clock.

Part I - Compressed-size counter (lines 277-295)

```
    // ---------------- compressed-size counter ----------------
    wire [11:0]        q_small  = q_val[11:0];
    wire               q_zero   = (q_small == 12'd0);
    wire signed [12:0] q_signed = q_neg ? -$signed({1'b0, q_small}) : $signed({1'b0, q_small});
    wire signed [13:0] dc_diff  = q_signed - prev_dc;
    wire [13:0]        dc_mag   = dc_diff[13] ? -dc_diff : dc_diff;
    wire [3:0]         dc_size  = nbits(dc_mag);
    wire [3:0]         ac_size  = nbits({2'b00, q_small});

    reg [9:0] bits_add;
    always @(*) begin
        if (cnt[5:0] == 6'd0)
            bits_add = 10'd4 + {6'd0, dc_size};                    // DC
        else if (q_zero)
            bits_add = (cnt[5:0] == 6'd63) ? 10'd8 : 10'd0;        // EOB at the end
        else
            bits_add = {5'd0, run[5:4], 3'b000}                    // ZRL per 16 zeros
                     + 10'd8 + {6'd0, ac_size};                    // symbol + amplitude
    end
```

This estimates how many bits a JPEG-style file would need, coefficient by coefficient:

Case	Bits added	JPEG meaning
DC (first coefficient)	4 + nbits(DC - previous DC)	DC is stored as the change from the previous block, a 4-bit size then the value
AC = 0, not the last	0	zeros are not stored one by one; they are counted in run
AC = 0 at position 63	8	end-of-block (EOB) symbol: 'the rest are all zero'
AC not zero	8 x (run / 16) + 8 + nbits(value)	ZRL symbols for each 16 zeros skipped, then one 8-bit (run, size) symbol and the value bits

run[5:4] is run / 16 and **{run[5:4], 3'b000}** is (run / 16) x 8. Real JPEG would code the 8-bit symbols with Huffman codes (shorter for common ones), so a real file would be about 20-30 % smaller still.

Part J - Output pixel and block limit (lines 297-302)

```
    // ---------------- output value (add 128, limit 0..255) ----------------
    wire signed [15:0] o_val   = bufA[cnt[5:0]] + 16'sd128;
    wire [7:0]         o_pixel = o_val[15]          ? 8'd0   :
                                 (o_val > 16'sd255) ? 8'd255 : o_val[7:0];

    wire [5:0] last_blk = is_y ? 6'd39 : 6'd19;       // 40x40 blocks for Y, 20x20 for Cb/Cr
```

- Adds 128 back (undoing the level shift) and clamps to 0..255, because rounding in the IDCT can produce values slightly outside the range (e.g. -2 or 258).
- **o_val[15]** is the sign bit: if set, the value is negative, so output 0.
- **last_blk** is 39 for Y (40 blocks per row) and 19 for Cb/Cr (20 blocks per row).

Part K - The control state machine (lines 304-422)

Everything above is datapath (wires that compute). The state machine decides **what happens on each clock**.

Initial values and START, lines 305-330

```
    initial begin
        state = S_START; chan = 0; mode = 0; cnt = 0; bx = 0; by = 0; shift = 0;
        acc = 0; wr_addr = 0; wr_data = 0; we = 0; wr_chan = 0; done = 0;
        prev_dc = 0; run = 0; total_bits = 0;
    end

    always @(posedge clk) begin
        we <= 1'b0;

        if (reset) begin
            state <= S_START;
            done  <= 1'b0;
        end else begin
            case (state)

            S_START: begin
                shift      <= quality;
                chan       <= 2'd0;
                bx         <= 6'd0;
                by         <= 6'd0;
                cnt        <= 9'd0;
                done       <= 1'b0;
                total_bits <= 24'd0;
                prev_dc    <= 13'sd0;
                state      <= S_LOAD;
            end
```

- **initial** sets power-up values (Quartus loads these into the FPGA when it is programmed).
- **we <= 0** at the top of the always block is a default: writing happens only in WRITE.
- **reset** sends the machine to START. **START** copies the quality switches, clears the counters, total_bits and prev_dc, and goes to LOAD.

LOAD, lines 332-351

```
            S_LOAD: begin
                if (is_y) begin                                     // Y: 64 pixels
                    if (cnt != 9'd0)
                        bufA[cnt_m1] <= y_val;
                    if (cnt == 9'd64) begin
                        cnt <= 9'd0; mode <= 2'd0; state <= S_MUL;
                    end else
                        cnt <= cnt + 9'd1;
                end else begin                                      // Cb/Cr: 256 pixels -> 64 samples
                    if (cnt != 9'd0) begin
                        if (!p_m1[4] && !p_m1[0])     bufA[c_idx] <= c_val;  // 1st of 2x2
                        else if (p_m1[4] && p_m1[0])  bufA[c_idx] <= c_avg;  // 4th -> average
                        else                          bufA[c_idx] <= c_acc;  // 2nd, 3rd
                    end
                    if (cnt == 9'd256) begin
                        cnt <= 9'd0; mode <= 2'd0; state <= S_MUL;
                    end else
                        cnt <= cnt + 9'd1;
                end
            end
```

- **Y**: store each arriving pixel at bufA[cnt-1]; after cnt = 64 all 64 are in.
- **Cb/Cr**: 256 pixels arrive; for each 2 x 2 group the 1st pixel is stored, the 2nd and 3rd are added, and the 4th is added and the sum divided by 4. Result: 64 averaged colour samples.
- Then mode = 0 and go to MUL.

MUL, lines 353-369

```
            S_MUL: begin
                acc <= acc_next;
                if (k == 3'd7) begin
                    if (mode[0] == 1'b0) bufB[{a, b}] <= mul_out;
                    else                 bufA[{a, b}] <= mul_out;
                end
                if (cnt == 9'd511) begin
                    cnt <= 9'd0;
                    case (mode)
                        2'd0: mode  <= 2'd1;
                        2'd1: state <= S_QUANT;
                        2'd2: mode  <= 2'd3;
                        2'd3: state <= S_WRITE;
                    endcase
                end else
                    cnt <= cnt + 9'd1;
            end
```

- Every clock: one multiply-add. When k = 7 the sum for output (a, b) is finished and stored (modes 0 and 2 write bufB; modes 1 and 3 write bufA, chosen by mode[0]).
- After 512 clocks: mode 0 -> mode 1; mode 1 -> QUANT (DCT done); mode 2 -> mode 3; mode 3 -> WRITE (IDCT done).

QUANT, lines 371-385

```
            S_QUANT: begin
                bufA[zz_n] <= dq_out;
                total_bits <= total_bits + {14'd0, bits_add};
                if (cnt[5:0] == 6'd0) begin
                    prev_dc <= q_signed;
                    run     <= 6'd0;
                end else if (q_zero)
                    run <= run + 6'd1;
                else
                    run <= 6'd0;
                if (cnt == 9'd63) begin
                    cnt <= 9'd0; mode <= 2'd2; state <= S_MUL;
                end else
                    cnt <= cnt + 9'd1;
            end
```

- Replace each coefficient with its quantized-then-dequantized value, add its bits to total_bits, update prev_dc (at the DC) and the zero run. After 64 clocks, mode = 2 and back to MUL for the IDCT.

WRITE, lines 387-396

```
            S_WRITE: begin
                we      <= 1'b1;
                wr_chan <= chan;
                wr_addr <= is_y ? y_wr_addr : c_wr_addr;
                wr_data <= o_pixel;
                if (cnt == 9'd63) begin
                    cnt <= 9'd0; state <= S_NEXT;
                end else
                    cnt <= cnt + 9'd1;
            end
```

- Write the 64 rebuilt values to the result memory for this channel, one per clock.

NEXT and DONE, lines 398-422

```
            S_NEXT: begin
                cnt   <= 9'd0;
                state <= S_LOAD;
                if (bx == last_blk) begin
                    bx <= 6'd0;
                    if (by == last_blk) begin
                        by <= 6'd0;
                        if (chan == 2'd2)
                            state <= S_DONE;                    // all 3 channels done
                        else begin
                            chan    <= chan + 2'd1;             // next channel
                            prev_dc <= 13'sd0;
                        end
                    end else
                        by <= by + 6'd1;
                end else
                    bx <= bx + 6'd1;
            end

            S_DONE: done <= 1'b1;

            default: state <= S_START;
            endcase
        end
    end
```

- Move right one block; at the end of a row go to the next row; after the last block of a channel move to the next channel and reset prev_dc (each channel has its own DC prediction, as in JPEG).
- After the last Cr block: DONE, which holds **done = 1** until the next reset.

3. Worked example: one plain block, start to finish

Take a Y block where all 64 pixels have brightness 200 (a flat area). These are the exact numbers the hardware produces (checked with a bit-accurate model of the code).

Step	What happens	Value
LOAD	200 - 128 (level shift)	all 64 entries of bufA = 72
DCT mode 0	T[0][j] = sum of 8 x (1448 x 72) / 4096, rounded	row 0 = 204, rows 1..7 = 0
DCT mode 1	F[0][0] = 8 x 204 x 1448 / 4096, rounded	F[0][0] = 577 (DC), all 63 AC = 0
QUANT	(577 x 4096 + 32768) >> 16 = 577 / 16	q = 36; dequantized = 36 x 16 = 576
Bits	DC: 4 + nbits(36 - 0) = 4 + 6; AC: 63 zeros -> EOB	10 + 8 = **18 bits** (original: 64 x 8 = 512 bits)
IDCT mode 2	576 x 1448 / 4096, rounded	column 0 = 204
IDCT mode 3	204 x 1448 / 4096, rounded	all 64 values = 72
WRITE	72 + 128	all 64 output pixels = 200 (identical to the input)

A flat block costs 18 bits instead of 512, and comes back perfectly. A block with sharp edges has many non-zero AC coefficients, costs more bits, and loses some detail in rounding. This is why the % saved depends on the picture, and why stronger quantization (SW1 SW0) makes the picture blocky.

4. Questions the professor may ask

Q: Why 8 x 8 blocks?

It is the JPEG standard. Small enough that the picture inside is roughly uniform and the maths is cheap, large enough to separate smooth areas from detail.

Q: Why subtract 128 before the DCT?

To centre the values on zero (-128..127). The DC coefficient then stays small for mid-gray and all values fit comfortably in 16 bits. It is part of the JPEG standard.

Q: Why do you compute the inverse DCT in a compressor?

To display the result. The right half of the screen shows what a JPEG decoder would rebuild from our quantized data, so the quality loss can be seen.

Q: Where exactly is the compression?

In QUANT. Dividing by the table and rounding makes most AC coefficients zero, and the bit counter shows that zeros cost almost nothing. The DCT itself loses nothing; it only rearranges the data.

Q: Why fixed-point instead of floating point?

Floating point needs large, slow hardware. Scaling constants by 4096 and shifting back gives enough accuracy (the worked example rebuilds the block exactly) using the FPGA's built-in multipliers.

Q: How do you divide without a divider?

Multiply by a pre-computed reciprocal (65536/Q) and shift right by 16. Address calculations like row x 320 use shifts and adds.

Q: Why zigzag order?

Low frequencies are visited first and high frequencies last. Since high frequencies are usually zero after quantization, the zeros form one long run at the end, which a single EOB symbol covers.

Q: Why are Cb and Cr half size?

Human eyes are much less sensitive to colour detail than to brightness detail. Halving both directions keeps 1/4 of the colour samples, saving 50 % of the total data before any DCT.

Q: Is this real JPEG?

It has the JPEG core: colour conversion, 4:2:0 subsampling, 8 x 8 DCT, standard quantization tables, zigzag and run-length symbols. It does not produce a .jpg file: we count fixed 8-bit symbols instead of Huffman coding, so the % saved is slightly lower than real JPEG.

Q: How long does one picture take, and could it be faster?

About 5.5 million cycles = 0.22 s at 25 MHz. It could be 8 times faster by computing the 8 products of each sum in parallel (8 multipliers instead of 1), or by running the DCT of one block while the next block loads.

Q: What does the quality switch do inside the code?

It sets shift, which multiplies every quantization step by 1, 2, 4 or 8. Bigger steps make more coefficients zero: smaller size, lower quality.

Q: Why two buffers (bufA and bufB)?

Each matrix multiply reads one buffer and writes the other, so it never overwrites data it still needs.